

Designated-initializers for Base Classes

Document #: P2287R0
Date: 2021-01-20
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Proposal](#)

[2.1 Lookup of base classes](#)

[3 Wording](#)

[4 References](#)

§ 1 Introduction

[[P0017R1](#)] extended aggregates to allow an aggregate to have a base class. [[P0329R4](#)] gave us designated initializers, which allow for much more expressive and functional initialization of aggregates. However, the two do not mix: a designated initializer can currently only refer to a direct non-static data members. This means that if I have a type like:

```
struct A {  
    int a;  
};  
  
struct B : A {  
    int b;  
};
```

While I can initialize an A like `A{.a=1}`, I cannot designated-initialize B. An attempt like `B{{1}, .b=2}` runs afoul of the rule that the initializers must either be all designated or none designated. But there is currently no way to designate the base class here.

Which means that my only options for initializing a B are to fall-back to regular aggregate initialization and write either `B{{1}, 2}` or `B{1, 2}`. Neither are especially satisfactory.

The goal of this paper is to extend designated initialization to include base classes.

§ 2 Proposal

The tricky part here is: how do we name the A base class of B in the *designated-initializer-list*? While non-static data members have *identifiers*, base classes can be much more complicated. They can be qualified names, they can have template arguments, etc. We also do not actually have a way to name the A base class subobject of a B today — the only way to get there is via a cast. This means there's no corresponding consistent syntax to choose along with the `.` that we already have.

Daveed Vandevoorde makes the suggestion that we can use `:` to introduce a *class-or-decltype* that names a base class (that is the grammar term we use when introducing a base class). This would allow the following initialization syntax:

```
B{:A={.a=1}, .b=2}
```

Using a `:` mimics the way we introduce base classes in class directions and is otherwise unambiguous with the rest of the *designated-initializer* syntax. It can also prepare the parser for the fact that a more complicated name might be coming.

This paper does not change any of the other existing designated-initialization rules: the initializers must still be all designated or none designated, and the designators must be in order. I'm simply extending the order being matched against with all the base classes. That is, while `B{:A={.a=1}, .b=2}` would be a valid way to initialize a B, `B{.b=2, :A={.a=1}}` is ill-formed (out of order), as is `B{{.a=1}, .b=2}` (some designated but not all).

This generalizes to more complex aggregates like:

```
template <typename T> struct C { T val; };
struct D : C<int>, C<char> { };

D{:C<int>={.val=1}, :C<char>={.val='x'}};
```

Which provides protection against `D{'x', 1}` which compiles fine today but probably isn't what was desired.

§ 2.1 Lookup of base classes

One other thing we need to consider is how we look up base classes exactly. With regular designated initializers, they're just the names of direct members and there's only one way to name them. Not much to talk about. But with base classes, we have an *injected-class-name* too, so we have to ask the question:

```
template <typename T> struct C { T val; };
struct D : C<int> { };

D{:C<int>{.val=0}}; // proposed okay, C<int> is a base class
D{:C{.val=1}};     // how about this?
```

From within the scope of D, we can use C to identify the base class `C<int>`. Likewise `D::C` names that type. Can we do this externally? Designated initializers already sort of look like they're from within the class. This seems like it should probably apply to base classes as well.

But in order for `:C` to find `D::C<int>::C` there, we'd have to say that lookup is in the context of the body of `D`. But then what if we have:

```
namespace N { template <typename T> struct C { }; }
struct D : N::C<int> { };
using C = N::C<double>;

D{:C{}};
```

Does `:C` refer to the *injected-class-name* of the `C<int>` base class of `D` (and thus be well-formed), or does `:C` refer to the alias `C` (and thus be ill-formed, since `N::C<double>` is not a base class of `D`)? Arguably, we are initializing a `D` so looking up from the context of `D` is a sensible rule (and is consistent with the rules we have for `x.operator T()` looking up `T` in the context of `x` 6.5.3 [basic.lookup.unqual]/5).

We just need to make sure that we don't do that kind of lookup for a *decltype-specifier* as a base class, since that probably makes very little sense to consider from the class' context.

§ 3 Wording

Change the grammar of a *designator* in 9.4.1 [dcl.init.general]/1. Technically this allows a *designated-initializer-list* like `{.a=1, :A={}}` which we could forbid grammatically, but that seems more complicated than simply extending the ordering rule to forbid it (which has to be done anyway).

```
designator:
    . identifier
+    : class-or-decltype
```

Extend 9.4.1 [dcl.init.general]/18:

- 18 The same *identifier* shall not appear in multiple designators of a *designated-initializer-list*.
The same *class-or-decltype* shall not appear in multiple designators of a *designated-initializer-list*.

Extend 9.4.2 [dcl.init.aggr]/3.1:

- (3.1) If the initializer list is a *designated-initializer-list*, the aggregate shall be of class type, the *identifier* in each designator shall name a direct non-static data member of the class, the *class-or-decltype* in each designator shall name a direct base class of the class, and the explicitly initialized elements of the aggregate are the elements that are, or contain, those members.

Extend 9.4.5 [dcl.init.list]/3.1:

- (3.1) If the *braced-init-list* contains a *designated-initializer-list*, τ shall be an aggregate class. The ordered *class-or-decltypes* and *identifiers* in the designators of the *designated-initializer-list* shall form a subsequence of the ordered *direct base classes of τ* and *identifiers* in the direct non-static data members of τ . Aggregate initialization is performed ([dcl.init.aggr]).
[Example 2:

```

struct A { int x; int y; int z; };
A a{.y = 2, .x = 1};           // error: designator order does not
    match declaration order
A b{.x = 1, .z = 2};           // OK, b.y initialized to 0

+ struct B : A { int q; };
+ B c{.q = 3, :A{}};           // error: designator order does not
    match declaration order
+ B d{:A{}, .q = 3};           // OK, d.x, d.y, and d.z all
    initialized to 0

```

— *end example*]

§ 4 References

[P0017R1] Oleg Smolsky. 2015-10-24. Extension to aggregate initialization.

<https://wg21.link/p0017r1>

[P0329R4] Tim Shen, Richard Smith. 2017-07-12. Designated Initialization Wording.

<https://wg21.link/p0329r4>